ChatGPT Conversation

URL: https://chatgpt.com/share/68acc9f9-fac0-800f-8804-1b16a3b1ba69

Date: 8/25/2025, 4:47:12 PM

Generated by chatgptopdf.in

Got it You want a **feature-based modular navigation structure** for a **Jetpack Compose Multiplatform (CMP) app**, with:

- **Unauthorized routes** (Splash, Login, Registration, etc.)
- **MainScreen** as the root after login → contains **BottomBar with 4 tabs**
- Each **tab has its own inner navigation graph** (subroutine-like flow)
- **Logout resets navigation** back to unauthorized graph
- Each feature/module defines its own **NavBuilder**

Here's how I'd **structure the navigation with an arrow-style map** (before writing code):

Navigation Arrow Map

Copy code

```
                    ┌──────────────────────────┐
                    │        Root NavHost      │
                    └──────────────────────────┘
                                 │
        ┌────────────────────────┴────────────────────────┐
        │                                                 │
        ▼                                                 ▼
┌──────────────┐                                  ┌──────────────┐
│Unauthorized Graph                               │Authorized Graph
│(Auth Feature Module)                            │(Main Feature Module)
└──────────────┘                                  └──────────────┘
        │                                                 │
        ▼                                                 ▼
┌──────────────┐                                  ┌──────────────┐
│ SplashScreen │                                  │ MainScreen   │
├──────────────┤                                  │ (Bottom Bar) │
│ LoginScreen  │                                  └──────────────┘
│ RegisterScreen │                                       │   │
└──────────────┘                  Logout ────────────────┘   │
        │                                                    │
        └──────────────────────────────► Back to Unauthorized │
                                                             │
```

Generated via chatgptopdf.in

```
┌──────────────────────────────────────────────────────────────────────┐
│                                                                      │
│                                                                      │
│                                                                      │
```

📁 Project Structure (Modules)

Copy code

```
app/                        # Root NavHost
core/                       # Shared utils, DI setup
features/
   auth/                    # Unauthorized routes
      ui/
      navigation/
         AuthNavGraph.kt
   main/                    # Authorized main screen + bottom bar
      ui/
      navigation/
         MainNavGraph.kt
   home/                    # Tab 1
      ui/
      navigation/
         HomeNavGraph.kt
   dashboard/               # Tab 2
      ui/
      navigation/
         DashboardNavGraph.kt
   profile/                 # Tab 3
      ui/
      navigation/
         ProfileNavGraph.kt
   settings/                # Tab 4
      ui/
      navigation/
         SettingsNavGraph.kt
```

🧭 Root NavHost

kotlin

Copy code

```kotlin
@Composable
fun RootNavHost(navController: NavHostController) {
    NavHost(
        navController = navController,
        startDestination = "auth"
    ) {
        authNavGraph(navController)
        mainNavGraph(navController)
    }
}
```

🔐 Auth Graph (Unauthorized)

kotlin

Copy code

```
fun NavGraphBuilder.authNavGraph(navController: NavHostController) {
    navigation(
        route = "auth",
        startDestination = "splash"
    ) {
        composable("splash") {
            SplashScreen(
                onNavigateToLogin = { navController.navigate("login") }
            )
        }
        composable("login") {
            LoginScreen(
                onLoginSuccess = {
                    navController.navigate("main") {
                        popUpTo("auth") { inclusive = true } // clear back stack
                    }
                }
            )
        }
        composable("register") {
            RegisterScreen(
                onRegisterSuccess = {
                    navController.navigate("main") {
                        popUpTo("auth") { inclusive = true }
                    }
                }
            )
        }
    }
}
```

Main Graph (Authorized with Bottom Bar)

kotlin

Copy code

```
fun NavGraphBuilder.mainNavGraph(navController: NavHostController) {
    navigation(
        route = "main",
        startDestination = "mainRoot"
    ) {
        composable("mainRoot") {
            MainScreen(
                onLogout = {
                    navController.navigate("auth") {
                        popUpTo("main") { inclusive = true } // reset to auth
                    }
                }
            )
        }
    }
}
```

MainScreen with Bottom Navigation

Generated via chatgptopdf.in

kotlin

Copy code

```kotlin
@Composable
fun MainScreen(onLogout: () -> Unit) {
    val navController = rememberNavController()
    val bottomItems = listOf("home", "dashboard", "profile", "settings")

    Scaffold(
        bottomBar = {
            BottomNavigation {
                bottomItems.forEach { screen ->
                    BottomNavigationItem(
                        icon = { Icon(Icons.Default.Home, contentDescription = nul
                        label = { Text(screen) },
                        selected = false,
                        onClick = {
                            navController.navigate(screen) {
                                popUpTo(navController.graph.findStartDestination()
                                    saveState = true
                                }
                                launchSingleTop = true
                                restoreState = true
                            }
                        }
                    )
                }
            }
        }
    ) { innerPadding ->
        NavHost(
            navController = navController,
            startDestination = "home",
            modifier = Modifier.padding(innerPadding)
        ) {
            homeNavGraph(navController)
            dashboardNavGraph(navController)
            profileNavGraph(navController, onLogout)
            settingsNavGraph(navController)
        }
    }
}
```

Example Tab Graph (Home Feature)

kotlin

Copy code

```kotlin
fun NavGraphBuilder.homeNavGraph(navController: NavHostController) {
    navigation(
        route = "home",
        startDestination = "home/main"
    ) {
        composable("home/main") {
            HomeScreen(
                onDetailClick = { navController.navigate("home/detail") }
```

```
            )
        }
        composable("home/detail") {
            DetailScreen(onBack = { navController.popBackStack() })
        }
    }
}
```

Example Tab Graph (Profile Feature with Logout)

kotlin

Copy code

```kotlin
fun NavGraphBuilder.profileNavGraph(
    navController: NavHostController,
    onLogout: () -> Unit
) {
    navigation(
        route = "profile",
        startDestination = "profile/main"
    ) {
        composable("profile/main") {
            ProfileScreen(
                onEditClick = { navController.navigate("profile/edit") },
                onLogout = onLogout
            )
        }
        composable("profile/edit") {
            EditProfileScreen(onBack = { navController.popBackStack() })
        }
    }
}
```

With this setup:

- **Feature modules** define their own graphs (`authNavGraph`, `homeNavGraph`, etc.)
- **RootNavHost** ties them together
- **Logout clears stack → back to Auth**
- **Bottom bar** manages tab navigation with its own NavHost

Do you want me to also **add DI setup with Koin** so that each `NavGraphBuilder` can inject its `ViewModels` independently, or should we keep DI separate for now?

Exactly you nailed it — **hardcoding route strings is risky** (typos, refactoring pain). The clean way is to **wrap each feature's routes in a sealed class (or enum)** so navigation is **type-safe**.

Let me show you how we can refactor the previous structure using **sealed classes per graph**.

---

1. Base NavRoute

```kotlin
sealed class NavRoute(val route: String) {
    object Root : NavRoute("root")
}
```

---

2. Auth Graph Routes

```kotlin
sealed class AuthRoute(route: String) : NavRoute(route) {
    object Graph : AuthRoute("auth")
    object Splash : AuthRoute("auth/splash")
    object Login : AuthRoute("auth/login")
    object Register : AuthRoute("auth/register")
}
```

---

3. Main Graph Routes

```kotlin
sealed class MainRoute(route: String) : NavRoute(route) {
    object Graph : MainRoute("main")
    object Root : MainRoute("main/root")
}
```

---

4. Home Graph Routes

```kotlin
sealed class HomeRoute(route: String) : NavRoute(route) {
    object Graph : HomeRoute("home")
    object Main : HomeRoute("home/main")
    object Detail : HomeRoute("home/detail")
}
```

---

5. Profile Graph Routes

Generated via chatgptopdf.in

kotlin

Copy code

```kotlin
sealed class ProfileRoute(route: String) : NavRoute(route) {
    object Graph : ProfileRoute("profile")
    object Main : ProfileRoute("profile/main")
    object Edit : ProfileRoute("profile/edit")
}
```

(Same idea for `DashboardRoute`, `SettingsRoute`)

---

6. Usage in NavGraphBuilder

Now, instead of `"auth"` or `"home/main"` strings, we use:

kotlin

Copy code

```kotlin
fun NavGraphBuilder.authNavGraph(navController: NavHostController) {
    navigation(
        route = AuthRoute.Graph.route,
        startDestination = AuthRoute.Splash.route
    ) {
        composable(AuthRoute.Splash.route) {
            SplashScreen(
                onNavigateToLogin = { navController.navigate(AuthRoute.Login.route
            )
        }
        composable(AuthRoute.Login.route) {
            LoginScreen(
                onLoginSuccess = {
                    navController.navigate(MainRoute.Graph.route) {
                        popUpTo(AuthRoute.Graph.route) { inclusive = true }
                    }
                }
            )
        }
        composable(AuthRoute.Register.route) {
            RegisterScreen(
                onRegisterSuccess = {
                    navController.navigate(MainRoute.Graph.route) {
                        popUpTo(AuthRoute.Graph.route) { inclusive = true }
                    }
                }
            )
        }
    }
}
```

---

7. Bottom Navigation Items

Instead of using `"home"`, `"dashboard"`, etc., we do:

kotlin

Copy code

```kotlin
val bottomItems = listOf(
    HomeRoute.Graph,
    DashboardRoute.Graph,
    ProfileRoute.Graph,
    SettingsRoute.Graph
)

BottomNavigation {
    bottomItems.forEach { route ->
        BottomNavigationItem(
            icon = { Icon(Icons.Default.Home, contentDescription = null) },
            label = { Text(route.route.substringAfter("/")) },
            selected = false,
            onClick = {
                navController.navigate(route.route) {
                    popUpTo(navController.graph.findStartDestination().id) {
                        saveState = true
                    }
                    launchSingleTop = true
                    restoreState = true
                }
            }
        )
    }
}
```

 Benefits:

- No hardcoded strings anywhere
- Each feature owns its **sealed class of routes**
- Compiler catches mistakes when navigating
- Easier refactor (rename a route → single source of truth)

 Would you like me to **go one step further** and make these `NavRoute` sealed classes also hold **typed arguments (with safe parsing)**, similar to how `NavType` works in Compose navigation? That way you won't even need to manually handle query params.

Great question you're aiming for a **clean, scalable navigation + DI setup** where:

- Unauthorized graph and authorized (bottom bar) graph each have their own **`NavController`**
- Both controllers are **provided down the tree via `CompositionLocal`**
- Each screen gets its **own ViewModel, injected via Koin** (or DI of choice)

Generated via chatgptopdf.in

- ▯ ViewModels are bound to **navigation lifecycle**, so they are cleared when the screen is popped

This is exactly how Android's Navigation + ViewModel integration works, but since we're in **Compose Multiplatform**, we'll wire it manually.

▯ Step 1. CompositionLocal for NavControllers

kotlin
Copy code
```kotlin
val LocalRootNavController = staticCompositionLocalOf<NavHostController> {
    error("Root NavController not provided")
}

val LocalBottomNavController = staticCompositionLocalOf<NavHostController?> {
    null // only available after login
}
```

▯ Step 2. Root Setup with Providers

kotlin
Copy code
```kotlin
@Composable
fun AppRoot() {
    val rootNavController = rememberNavController()

    CompositionLocalProvider(
        LocalRootNavController provides rootNavController,
        LocalBottomNavController provides null
    ) {
        RootNavHost(rootNavController)
    }
}
```

▯ Step 3. MainScreen with Bottom Nav Controller

kotlin
Copy code
```kotlin
@Composable
fun MainScreen(onLogout: () -> Unit) {
    val bottomNavController = rememberNavController()

    CompositionLocalProvider(
        LocalBottomNavController provides bottomNavController
    ) {
        Scaffold(
            bottomBar = { BottomBar(bottomNavController) }
        ) { innerPadding ->
```

Generated via chatgptopdf.in

```
            NavHost(
                navController = bottomNavController,
                startDestination = HomeRoute.Graph.route,
                modifier = Modifier.padding(innerPadding)
            ) {
                homeNavGraph(bottomNavController)
                dashboardNavGraph(bottomNavController)
                profileNavGraph(bottomNavController, onLogout)
                settingsNavGraph(bottomNavController)
            }
        }
    }
}
```

Step 4. ViewModel Injection per Screen

Since you're using **Koin**, we can use `koinViewModel()` (CMP friendly alternative exists).
We want the VM to be scoped to the **screen lifecycle**, not global.

kotlin
Copy code
```
@Composable
fun HomeScreen(
    onDetailClick: () -> Unit,
    viewModel: HomeViewModel = koinViewModel()
) {
    val state by viewModel.state.collectAsState()

    Column {
        Text("Home: ${state.title}")
        Button(onClick = onDetailClick) { Text("Go to Detail") }
    }
}
```

Step 5. ViewModel Cleared on Pop

Koin provides `viewModel {}` scoped to navigation graph lifecycle **if** you integrate with `NavBackStackEntry`.
We can mimic **Hilt's SavedStateHandle / NavBackStackEntry binding** in CMP with an extension:

kotlin
Copy code
```
@Composable
inline fun <reified VM : ViewModel> NavBackStackEntry.koinViewModel(): VM {
    return getKoin().getViewModel(
        key = id, // unique per destination
        clazz = VM::class
```

```
    )
}
```

Then in your `composable`:

```kotlin
composable(HomeRoute.Main.route) { backStackEntry ->
    val vm: HomeViewModel = backStackEntry.koinViewModel()
    HomeScreen(onDetailClick = { navController.navigate(HomeRoute.Detail.route) },
}
```

When this `backStackEntry` is popped, the VM is disposed

Step 6. Scoped Modules per Feature

In each feature module:

```kotlin
val homeModule = module {
    viewModel { HomeViewModel(get()) }
}
```

Register in your `appModule`:

```kotlin
startKoin {
    modules(
        listOf(
            coreModule,
            authModule,
            mainModule,
            homeModule,
            dashboardModule,
            profileModule,
            settingsModule
        )
    )
}
```

So now:

- `RootNavController` → available everywhere via `LocalRootNavController`
- `BottomNavController` → available only after login via `LocalBottomNavController`
- Each screen uses **feature-specific VM**, scoped to its **NavBackStackEntry**

Generated via chatgptopdf.in

- When the user **pops a screen / logs out**, its VM is destroyed automatically

---

 Do you want me to also show how to **pass arguments safely** into these sealed `NavRoute` classes (like `ProfileRoute.Detail(userId: String)`), so the VM can use typed args from navigation?

Ahh got it — thanks for clarifying.
Since you're building **Compose Multiplatform (CMP) with shared UI**, you can't rely on **Android-only `viewModel{}` + NavBackStackEntry integration**.
Instead, you need a **pure Compose solution** that works across platforms (Android, iOS, Desktop).

That means:

- No `viewModel {}` DSL from AndroidX or Hilt/Koin Android
- Instead, use **Koin `factory {}` or `single {}`** for DI
- Bind lifecycle manually using **`rememberSaveable` + `DisposableEffect`** so when a screen is popped, the VM (or presenter) is disposed

---

Step 1. Base "Screen Scoped" ViewModel

In CMP you usually implement your own VM base with `CoroutineScope` + `close()`:

kotlin
Copy code
```kotlin
abstract class ScreenViewModel {
    protected val job = SupervisorJob()
    protected val scope = CoroutineScope(Dispatchers.Main + job)

    open fun onCleared() {
        job.cancel() // cancel coroutines
    }
}
```

Step 2. Example Feature ViewModel

kotlin
Copy code
```kotlin
class HomeViewModel(
    private val repo: HomeRepository
) : ScreenViewModel() {

    private val _state = MutableStateFlow("Initial Home State")
    val state: StateFlow<String> = _state.asStateFlow()
```

```
    init {
        scope.launch {
            _state.value = repo.getData()
        }
    }
}
```

Step 3. Provide ViewModel in a `composable`

We create a helper that **remembers VM per NavBackStackEntry** and clears it when the entry is popped:

kotlin
Copy code
```kotlin
@Composable
inline fun <reified VM : ScreenViewModel> rememberScreenViewModel(
    key: String,
    crossinline factory: () -> VM
): VM {
    val vm = rememberSaveable(key = key) { factory() }

    DisposableEffect(Unit) {
        onDispose {
            if (vm is ScreenViewModel) {
                vm.onCleared()
            }
        }
    }
    return vm
}
```

Step 4. Usage in Navigation Graph

kotlin
Copy code
```kotlin
composable(HomeRoute.Main.route) { backStackEntry ->
    val vm = rememberScreenViewModel(key = backStackEntry.id) {
        getKoin().get<HomeViewModel>() // Koin factory { HomeViewModel(get()) }
    }

    HomeScreen(
        onDetailClick = { navController.navigate(HomeRoute.Detail.route) },
        viewModel = vm
    )
}
```

Here:

- `backStackEntry.id` ensures each screen instance has a unique key

- When popped, `onDispose` calls `vm.onCleared()` → coroutine scope is cancelled → VM is garbage collected

---

Step 5. Screen Composable Example

```kotlin
@Composable
fun HomeScreen(
    onDetailClick: () -> Unit,
    viewModel: HomeViewModel
) {
    val state by viewModel.state.collectAsState()

    Column {
        Text("Home State: $state")
        Button(onClick = onDetailClick) { Text("Go Detail") }
    }
}
```

Step 6. Koin Module (Cross-platform Safe)

Since we can't use `viewModel {}` in shared code, we define VMs as **factory**:

```kotlin
val homeModule = module {
    factory { HomeViewModel(get()) }  // create new per request
    single { HomeRepository() }
}
```

This works in Android, iOS, Desktop since it's just Kotlin.

Now you have:

- A **CompositionLocal** for NavControllers (root + bottom)
- A **ScreenViewModel** base that is cross-platform safe
- A **rememberScreenViewModel** helper bound to `NavBackStackEntry` lifecycle
- **Koin factory { }** instead of `viewModel { }`

So when a screen is **pushed → VM created**, when it's **popped → VM cleared**.

Do you want me to **extend this sealed NavRoute setup** to support **typed arguments** (e.g., `ProfileRoute.Detail(userId: String)`) so your `rememberScreenViewModel` can automatically get those args too?

Generated via chatgptopdf.in

Perfect let's carefully **break down that `rememberScreenViewModel` function line by line** so it's crystal clear how it works and why each part is there.

Function Signature

```kotlin
@Composable
inline fun <reified VM : ScreenViewModel> rememberScreenViewModel(
    key: String,
    crossinline factory: () -> VM
): VM
```

- `@Composable` → can only be called inside a Compose function.
- `inline` + `reified VM` → allows us to use the **generic type `VM` at runtime** (we know the actual type of ViewModel).
- `key: String` → a **unique identifier** for this VM instance (usually `backStackEntry.id` or route string). Ensures correct scoping.
- `factory: () -> VM` → how to create a new VM if one does not exist yet (e.g., via Koin `get()` or manual constructor).

The function **returns a VM instance** that survives recompositions but is disposed when the navigation entry is popped.

Inside the Function

1. Creating / Remembering VM

```kotlin
val vm = rememberSaveable(key = key) { factory() }
```

- `rememberSaveable` → keeps the object **alive across recompositions and configuration changes** (rotation, window resize).
- `key` → ensures uniqueness. If you navigate to the same screen multiple times, each gets its own VM because the key differs.
- `factory()` → creates a new VM **only the first time** (or when key changes).

 Without `rememberSaveable`, your VM would be recreated on every recomposition → bad for stateful objects.

Generated via chatgptopdf.in

2. Lifecycle Cleanup

```kotlin
Copy code
DisposableEffect(Unit) {
    onDispose {
        if (vm is ScreenViewModel) {
            vm.onCleared()
        }
    }
}
```

- `DisposableEffect(Unit)` → runs **once when the Composable enters the composition**, and its `onDispose` runs when the Composable **leaves composition** (e.g., screen is popped).
- Inside `onDispose` → call `vm.onCleared()`, which cancels its coroutines / cleans up resources.
- This mimics Android's **ViewModel lifecycle** (`onCleared()` called when destroyed).

 This is what ensures **your VM doesn't leak memory** after leaving the navigation stack.

---

3. Return VM

```kotlin
Copy code
return vm
```

- Returns the remembered VM to the Composable that called this helper.
- Since VM is tied to the Composable lifecycle, each screen gets exactly **one VM instance**, and it's cleaned up when the screen is removed.

How it Works Together

1. You navigate to a screen → `rememberScreenViewModel(key, factory)` runs
    - If VM exists for this key → reuse it
    - If not → create a new one with `factory()`
2. Screen recomposes many times → VM stays the same (thanks to `rememberSaveable`).
3. You pop the screen → Composable leaves composition → `onDispose` runs → `vm.onCleared()` cleans up resources.

Generated via chatgptopdf.in

Example in Practice

```kotlin
Copy code
composable(HomeRoute.Main.route) { backStackEntry ->
    val vm = rememberScreenViewModel(key = backStackEntry.id) {
        HomeViewModel(get()) // DI factory
    }

    HomeScreen(viewModel = vm)
}
```

- `backStackEntry.id` → ensures that each screen in the stack gets its own VM.
- VM is tied to that **nav entry's lifecycle**.
- When you pop back → VM is cleared.

This is a **platform-agnostic ViewModel scoping system** for CMP:

- Works on Android, iOS, Desktop
- No reliance on AndroidX `viewModelStoreOwner`
- Cleans up automatically

Would you like me to **improve this further** so that `rememberSaveable` can also **restore state** (like `SavedStateHandle` in Android) when navigating back to a screen? That way, your VM can survive process death or tab switching too.

Great let's extend our **`rememberScreenViewModel`** so it also supports **state restoration** — similar to Android's `SavedStateHandle`.

That way:

- If you navigate **away and back**, your screen state is restored.
- If the app is **killed and recreated** (on Android), the state survives.
- This still works in **shared UI (CMP)** without Android dependencies.

Step 1. Add State Saving Capability to ViewModel

We'll let each `ScreenViewModel` define how to **save and restore state**.

```kotlin
Copy code
```

```
abstract class ScreenViewModel {
    protected val job = SupervisorJob()
    protected val scope = CoroutineScope(Dispatchers.Main + job)

    // Called when saving state
    open fun saveState(): Map<String, Any?> = emptyMap()

    // Called when restoring state
    open fun restoreState(state: Map<String, Any?>) {}

    open fun onCleared() {
        job.cancel()
    }
}
```

Now each VM can override `saveState()` and `restoreState()` if it has something to persist.

Step 2. Extend `rememberScreenViewModel`

kotlin
Copy code

```
@Composable
inline fun <reified VM : ScreenViewModel> rememberScreenViewModel(
    key: String,
    crossinline factory: () -> VM
): VM {
    // Hold saved state across process death
    val saver = remember {
        mapSaver(
            save = { vm -> vm.saveState() },
            restore = { restoredState ->
                factory().apply { restoreState(restoredState) }
            }
        )
    }

    val vm = rememberSaveable(saver = saver, key = key) { factory() }

    DisposableEffect(Unit) {
        onDispose {
            vm.onCleared()
        }
    }

    return vm
}
```

Key points:

- We use `mapSaver`:
  - `save = { vm.saveState() }` → calls VM's save function before disposal or process death

- `restore = { factory().apply { restoreState(restoredState) } }` → creates a new VM and injects restored state

- This mimics `SavedStateHandle` but works cross-platform.

Step 3. Example ViewModel with State

```kotlin
class CounterViewModel : ScreenViewModel() {
    private val _count = MutableStateFlow(0)
    val count: StateFlow<Int> = _count

    fun increment() {
        _count.value += 1
    }

    override fun saveState(): Map<String, Any?> {
        return mapOf("count" to _count.value)
    }

    override fun restoreState(state: Map<String, Any?>) {
        (state["count"] as? Int)?.let { _count.value = it }
    }
}
```

Step 4. Usage in Navigation

```kotlin
composable("counter") { backStackEntry ->
    val vm = rememberScreenViewModel(key = backStackEntry.id) { CounterViewModel()

    CounterScreen(viewModel = vm)
}
```

Step 5. Example Screen

```kotlin
@Composable
fun CounterScreen(viewModel: CounterViewModel) {
    val count by viewModel.count.collectAsState()

    Column {
        Text("Count = $count")
        Button(onClick = { viewModel.increment() }) {
            Text("Increment")
```

Generated via chatgptopdf.in

```
        }
    }
}
```

- Navigate away → back → **state survives**
- Process death (Android) → **state restored**
- On iOS/Desktop → works because `rememberSaveable` persists inside Compose runtime

---

 Now your CMP navigation has:

- **CompositionLocal NavControllers** (root + bottom bar)
- **ScreenViewModels** with cleanup on pop (`onCleared()`)
- **State persistence** like `SavedStateHandle`, but multiplatform

 Do you want me to now extend this into your **sealed route classes** so that arguments (like `ProfileRoute.Detail(userId: String)`) also get **saved & restored automatically** into the VM state?

Awesome now let's take the **sealed route classes** we created earlier and make them support **typed arguments + automatic save/restore** so they integrate nicely with our `rememberScreenViewModel`.

This way:

- Navigation is **type-safe** (no hardcoded strings).
- Arguments are **encoded/decoded** into route strings.
- Arguments are **restored** after process death or when returning to a screen.

Step 1. Extend `NavRoute` Base Class

We'll allow `NavRoute` to define optional **arguments**:

kotlin
Copy code
```kotlin
sealed class NavRoute(val route: String) {
    open val args: List<NamedNavArgument> = emptyList()

    // Helper to build route with arguments
    fun withArgs(vararg values: Pair<String, Any?>): String {
        var newRoute = route
        values.forEach { (key, value) ->
            newRoute += "?$key=$value"
```

```
        }
        return newRoute
    }
}
```

Step 2. Example Route with Argument

For profile detail screen:

```kotlin
sealed class ProfileRoute(route: String) : NavRoute(route) {
    object Graph : ProfileRoute("profile")
    object Main : ProfileRoute("profile/main")
    data class Detail(val userId: String) : ProfileRoute("profile/detail") {
        override val args = listOf(
            navArgument("userId") { type = NavType.StringType }
        )
    }
}
```

- `ProfileRoute.Detail("123")` → route string becomes `profile/detail?userId=123`.
- `args` tells navigation that `userId` is expected.

Step 3. Use Route in NavGraph

```kotlin
fun NavGraphBuilder.profileNavGraph(
    navController: NavHostController,
    onLogout: () -> Unit
) {
    navigation(
        route = ProfileRoute.Graph.route,
        startDestination = ProfileRoute.Main.route
    ) {
        composable(ProfileRoute.Main.route) { backStackEntry ->
            val vm = rememberScreenViewModel(key = backStackEntry.id) { ProfileVie
            ProfileScreen(
                onEditClick = { navController.navigate(ProfileRoute.Detail("123").
                onLogout = onLogout,
                viewModel = vm
            )
        }

        composable(
            route = ProfileRoute.Detail("").route + "?userId={userId}",
            arguments = ProfileRoute.Detail("").args
        ) { backStackEntry ->
            val userId = backStackEntry.arguments?.getString("userId") ?: ""
```

Generated via chatgptopdf.in

```
            val vm = rememberScreenViewModel(key = backStackEntry.id) {
                ProfileDetailViewModel(userId)
            }
            ProfileDetailScreen(viewModel = vm)
        }
    }
}
```

Step 4. ViewModel that Uses Arguments

```kotlin
class ProfileDetailViewModel(private val userId: String) : ScreenViewModel() {
    private val _state = MutableStateFlow("Loading user $userId")
    val state: StateFlow<String> = _state

    override fun saveState(): Map<String, Any?> {
        return mapOf("userId" to userId)
    }

    override fun restoreState(state: Map<String, Any?>) {
        // no-op since we already pass userId in constructor
    }
}
```

Here:

- When navigating, `userId` gets passed into VM constructor.
- If process dies, `saveState()` stores it and `restoreState()` reloads it.

Step 5. Screen Example

```kotlin
@Composable
fun ProfileDetailScreen(viewModel: ProfileDetailViewModel) {
    val state by viewModel.state.collectAsState()
    Text("Profile Detail: $state")
}
```

Flow Recap

1. Define **typed routes** as sealed classes (`ProfileRoute.Detail(userId: String)`).
2. Navigation encodes arguments into the route string (`profile/detail?userId=123`).

3. `NavGraphBuilder` extracts arguments and passes to VM factory.
4. `rememberScreenViewModel` ensures VM is tied to that back stack entry.
5. VM can also **persist arguments** via `saveState()` across process death.

 Now your setup has:

- **Sealed routes** with typed arguments
- **Automatic VM lifecycle management** (creation + cleanup on pop)
- **State persistence** across recomposition & process death
- **Type-safe navigation** (no raw strings)

 Do you also want me to show a **generic helper function** so that when you define a sealed route like `ProfileRoute.Detail(userId)`, you can directly navigate with `navController.navigate(ProfileRoute.Detail("123"))` (without calling `.withArgs()` manually)?

Aha I see where you're going — you don't like "encoding arguments into route strings" (too Android-specific / brittle).

Since you're in **Compose Multiplatform** with **shared UI**, you want a **shared, DI-scoped way to pass arguments**:

- When you **navigate forward**, set arguments in a **shared ViewModel / store**.
- The destination screen **reads from that VM**.
- When you **pop back**, clear those arguments (so they don't leak).

This gives you **type-safety, avoids fragile route strings, and works on iOS/Desktop** as well.

Step 1. Shared ArgumentStore

We can use a simple `ViewModel`-like class:

kotlin
Copy code
```kotlin
class NavArgsStore {
    private val args = mutableMapOf<String, Any?>()

    fun <T> set(key: String, value: T) {
        args[key] = value
    }
```

```
    @Suppress("UNCHECKED_CAST")
    fun <T> get(key: String): T? = args[key] as? T

    fun clear(key: String) {
        args.remove(key)
    }
}
```

Register it in Koin as a `single { NavArgsStore() }`.

Step 2. Store + Clear Arguments When Navigating

Instead of encoding args in route strings:

kotlin
Copy code
```
fun NavController.navigateWithArgs(
    route: String,
    navArgsStore: NavArgsStore,
    argsKey: String,
    value: Any
) {
    navArgsStore.set(argsKey, value)
    this.navigate(route)
}
```

Step 3. Retrieve in Screen's ViewModel

kotlin
Copy code
```
class ProfileDetailViewModel(
    private val navArgsStore: NavArgsStore
) : ScreenViewModel() {

    private val _state = MutableStateFlow("")
    val state: StateFlow<String> = _state

    fun loadUser(argsKey: String) {
        val userId: String? = navArgsStore.get(argsKey)
        _state.value = "User detail for $userId"
    }

    override fun onCleared() {
        navArgsStore.clear("userId") // clean when screen exits
        super.onCleared()
    }
}
```

Step 4. NavGraph Usage

Generated via chatgptopdf.in

kotlin

Copy code

```kotlin
fun NavGraphBuilder.profileNavGraph(
    navController: NavHostController,
    navArgsStore: NavArgsStore
) {
    navigation(
        route = ProfileRoute.Graph.route,
        startDestination = ProfileRoute.Main.route
    ) {
        composable(ProfileRoute.Main.route) {
            val vm = rememberScreenViewModel(key = "profile_main") { ProfileViewMo
            ProfileScreen(
                onUserClick = { userId ->
                    navController.navigateWithArgs(
                        route = ProfileRoute.Detail.route,
                        navArgsStore = navArgsStore,
                        argsKey = "userId",
                        value = userId
                    )
                },
                viewModel = vm
            )
        }

        composable(ProfileRoute.Detail.route) { backStackEntry ->
            val vm = rememberScreenViewModel(key = backStackEntry.id) {
                ProfileDetailViewModel(navArgsStore).apply {
                    loadUser("userId")
                }
            }
            ProfileDetailScreen(viewModel = vm)
        }
    }
}
```

Step 5. Benefits

- No string-encoded query params.
- Works the same on Android, iOS, Desktop.
- Arguments **live in DI container (NavArgsStore)**, not in fragile route strings.
- Cleared automatically when VM dies → no memory leaks.

This pattern is very close to how **Flutter's Riverpod/Provider** does arguments passing.
You navigate, put values into a shared store, and the target screen's VM consumes them.

Do you want me to show how we can make this even **type-safe** (e.g. `ProfileArgs.UserId("123")`) instead of using string keys like `"userId"`?

Generated via chatgptopdf.in

Exactly — you caught it.

We don't *have* to do the explicit `loadUser()` step if the **ViewModel already has access to `NavArgsStore` via DI (Koin)**. That way:

- The VM can just **pull args on demand** whenever needed.
- You don't need to preload them in `init { }` or via a special `loadUser()` call.
- Clearing can be more defensive (`remove only if exists`).

Refined `NavArgsStore`

kotlin
Copy code
```kotlin
class NavArgsStore {
    private val args = mutableMapOf<String, Any?>()

    fun <T> set(key: String, value: T) {
        args[key] = value
    }

    @Suppress("UNCHECKED_CAST")
    fun <T> get(key: String): T? = args[key] as? T

    fun clear(key: String) {
        if (args.containsKey(key)) {
            args.remove(key)
        }
    }
}
```

ViewModel Example (Cleaner)

kotlin
Copy code
```kotlin
class ProfileDetailViewModel(
    private val navArgsStore: NavArgsStore
) : ScreenViewModel() {

    val userId: String?
        get() = navArgsStore.get("userId")

    override fun onCleared() {
        navArgsStore.clear("userId")
        super.onCleared()
```

Generated via chatgptopdf.in

```
    }
}
```

Now the VM doesn't need a `loadUser()` method. It just lazily pulls `userId` when a screen queries `viewModel.userId`.

Usage in Screen

```kotlin
Copy code
composable(ProfileRoute.Detail.route) { backStackEntry ->
    val vm = rememberScreenViewModel(key = backStackEntry.id) {
        ProfileDetailViewModel(get())
    }
    ProfileDetailScreen(viewModel = vm)
}
```

Benefits of This Refactor

1. **No preload step** (`loadUser()` unnecessary).
2. **Cleaner lifecycle binding** → VM owns the arg, clears it on `onCleared()`.
3. **Safer clear** → only removes if exists.
4. **DI-friendly** → Works the same across Android/iOS/Desktop since it's just a Kotlin class.

Do you want me to push this one step further and make the `NavArgsStore` **typed per-graph** (e.g. `ProfileArgsStore`) so you don't even use `"userId"` strings anywhere? That way each graph has its own strongly-typed args wrapper.

Generated via chatgptopdf.in